stat431_final report R implementation

Jack Liu (jackliu3), Xing Sun (xingsun3), Ning Xu (ningx4), Martin Wang(hanran2)

2026-05-12

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------------------------------------------
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.4
## -- Conflicts ------------------------------------------------------------------------------------
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
# Data cleaning for Bayesian hierarchical used-car price model


library(tidyverse)

find_project_root <- function() {
  for (start in c(normalizePath(".", mustWork = FALSE), normalizePath("..", mustWork = FALSE))) {
    path <- start
    for (i in 1:5) {
      if (dir.exists(file.path(path, "data", "raw"))) {
        return(normalizePath(path))
      }
      parent <- dirname(path)
      if (identical(parent, path)) {
        break
      }
      path <- parent
    }
  }
  stop("Could not find final_project root (expected data/raw/).")
}

ROOT <- find_project_root()
p <- function(...) file.path(ROOT, ...)

for (subdir in c(
  "data/processed",
  "models/jags",
  "results/mcmc",
  "results/prior_sensitivity",
```

```
  "results/model_comparison",
  "results/prediction",
  "results/eda"
)) {
  dir.create(p(subdir), recursive = TRUE, showWarnings = FALSE)
}

# 1. Read raw data

INPUT_PATH <- p("data/raw/used_cars.csv")

car_raw <- read_csv(
  INPUT_PATH,
  na = c("", "NA", "N/A", "NaN", "nan", "null", "NULL"),
  show_col_types = FALSE
)

cat("Raw data dimensions:\n")
```

```
## Raw data dimensions:
```

```
print(dim(car_raw))
```

```
## [1] 4009   12
```

```
cat("\nColumn names:\n")
```

```
##
## Column names:
```

```
print(names(car_raw))
```

```
##  [1] "brand"        "model"        "model_year"   "milage"       "fuel_type"
##  [6] "engine"       "transmission" "ext_col"      "int_col"      "accident"
## [11] "clean_title"  "price"
```

```
cat("\nFirst few rows:\n")
```

```
##
## First few rows:
```

```
print(head(car_raw))
```

```
## # A tibble: 6 x 12
##   brand    model model_year milage fuel_type engine transmission ext_col int_col
##   <chr>    <chr>      <dbl> <chr>  <chr>     <chr>  <chr>        <chr>   <chr>
## 1 Ford     Util~       2013 51,00~ E85 Flex~ 300.0~ 6-Speed A/T  Black   Black
## 2 Hyundai  Pali~       2021 34,74~ Gasoline  3.8L ~ 8-Speed Aut~ Moonli~ Gray
## 3 Lexus    RX 3~       2022 22,37~ Gasoline  3.5 L~ Automatic    Blue    Black
## 4 INFINITI Q50 ~       2015 88,90~ Hybrid    354.0~ 7-Speed A/T  Black   Black
## 5 Audi     Q3 4~       2021 9,835~ Gasoline  2.0L ~ 8-Speed Aut~ Glacie~ Black
## 6 Acura    ILX ~       2016 136,3~ Gasoline  2.4 L~ F            Silver  Ebony.
## # i 3 more variables: accident <chr>, clean_title <chr>, price <chr>
```

```
missing_before <- car_raw %>%
  summarise(across(everything(), ~sum(is.na(.)))) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "missing_count")
```

```r
cat("\nMissing values before cleaning:\n")
```

```
##
## Missing values before cleaning:
```

```r
print(missing_before)
```

```
## # A tibble: 12 x 2
##    variable     missing_count
##    <chr>                <int>
##  1 brand                    0
##  2 model                    0
##  3 model_year               0
##  4 milage                   0
##  5 fuel_type              170
##  6 engine                   0
##  7 transmission             0
##  8 ext_col                  0
##  9 int_col                  0
## 10 accident               113
## 11 clean_title            596
## 12 price                    0
```

```r
# 2. Parse numeric variables

car_parsed <- car_raw %>%
  mutate(
    price_num = parse_number(price),
    mileage_num = parse_number(milage),
    model_year_num = as.integer(model_year),

    brand = str_squish(as.character(brand)),
    model = str_squish(as.character(model)),
    fuel_type = str_squish(as.character(fuel_type)),
    transmission = str_squish(as.character(transmission)),
    accident = str_squish(as.character(accident)),
    clean_title = str_squish(as.character(clean_title))
  )

# 3. Remove impossible or extreme rows
# price <= 200000 removes very high-end exotic listings.
# If you want to keep exotic cars, set USE_MAINSTREAM_FILTER <- FALSE.

USE_MAINSTREAM_FILTER <- TRUE
MAX_PRICE <- 200000
MAX_MILEAGE <- 300000
MIN_MODEL_YEAR <- 1980

car_filtered <- car_parsed %>%
  filter(
    !is.na(price_num),
    !is.na(mileage_num),
    !is.na(model_year_num),
    !is.na(brand),
    price_num > 0,
```

```
    mileage_num >= 0,
    model_year_num >= MIN_MODEL_YEAR,
    model_year_num <= max(model_year_num, na.rm = TRUE)
  )

if (USE_MAINSTREAM_FILTER) {
  car_filtered <- car_filtered %>%
    filter(
      price_num <= MAX_PRICE,
      mileage_num <= MAX_MILEAGE
    )
}

cat("\nRows after numeric parsing and filtering:\n")
```

```
##
## Rows after numeric parsing and filtering:
```

```
print(nrow(car_filtered))
```

```
## [1] 3928
```

```
cat("Rows removed from raw data:\n")
```

```
## Rows removed from raw data:
```

```
print(nrow(car_raw) - nrow(car_filtered))
```

```
## [1] 81
```

```
# 4. Feature engineering

REFERENCE_YEAR <- max(car_filtered$model_year_num, na.rm = TRUE)

car_features <- car_filtered %>%
  mutate(
    age = REFERENCE_YEAR - model_year_num,
    log_price = log(price_num),
    log_mileage = log(mileage_num + 1)
  )

cat("\nReference year used to define vehicle age:\n")
```

```
##
## Reference year used to define vehicle age:
```

```
print(REFERENCE_YEAR)
```

```
## [1] 2024
```

```
# 5. Recode categorical variables

car_recode <- car_features %>%
  mutate(
    fuel_type_clean = case_when(
      is.na(fuel_type) ~ "Unknown/Other",
      fuel_type %in% c("-", "-", "not supported", "Not Supported") ~ "Unknown/Other",
      TRUE ~ fuel_type
```

```
    ),

    transmission_clean = case_when(
      is.na(transmission) ~ "Unknown",
      str_detect(str_to_lower(transmission), "manual|m/t") ~ "Manual",
      str_detect(str_to_lower(transmission), "cvt") ~ "CVT",
      str_detect(str_to_lower(transmission), "automatic|a/t|dual shift|auto") ~ "Automatic",
      transmission %in% c("-", "–") ~ "Unknown",
      TRUE ~ "Other"
    ),

    accident_clean = case_when(
      is.na(accident) ~ "Unknown",
      str_detect(str_to_lower(accident), "at least|accident|damage") ~ "Accident/damage reported",
      str_detect(str_to_lower(accident), "none") ~ "None reported",
      TRUE ~ "Unknown"
    ),

    clean_title_clean = case_when(
      is.na(clean_title) ~ "Unknown",
      clean_title %in% c("Yes", "yes", "Y", "y") ~ "Yes",
      TRUE ~ "Unknown"
    )
  )

# 6. Collapse rare brands


BRAND_MIN_N <- 20

brand_counts <- car_recode %>%
  count(brand, sort = TRUE)

brands_keep <- brand_counts %>%
  filter(n >= BRAND_MIN_N) %>%
  pull(brand)

car_model <- car_recode %>%
  mutate(
    brand_group = if_else(brand %in% brands_keep, brand, "Other")
  )

cat("\nNumber of original brand levels:\n")
```

```
##
## Number of original brand levels:
```

```
print(n_distinct(car_recode$brand))
```

```
## [1] 56
```

```
cat("Number of brand levels after combining rare brands:\n")
```

```
## Number of brand levels after combining rare brands:
```

```r
print(n_distinct(car_model$brand_group))
```

```
## [1] 35
```

```r
cat("\nBrand counts after combining rare brands:\n")
```

```
##
## Brand counts after combining rare brands:
```

```r
print(car_model %>% count(brand_group, sort = TRUE))
```

```
## # A tibble: 35 x 2
##    brand_group       n
##    <chr>         <int>
##  1 Ford            382
##  2 BMW             375
##  3 Mercedes-Benz   306
##  4 Chevrolet       292
##  5 Audi            200
##  6 Toyota          199
##  7 Porsche         188
##  8 Lexus           163
##  9 Jeep            143
## 10 Land            130
## # i 25 more rows
```

```r
# 7. Keep final modeling variables

car_model <- car_model %>%
  select(
    price = price_num,
    log_price,
    model_year = model_year_num,
    age,
    mileage = mileage_num,
    log_mileage,
    brand,
    brand_group,
    fuel_type = fuel_type_clean,
    transmission = transmission_clean,
    accident = accident_clean,
    clean_title = clean_title_clean
  ) %>%
  mutate(
    brand_group = factor(brand_group),
    fuel_type = factor(fuel_type),
    transmission = factor(transmission),
    accident = factor(accident),
    clean_title = factor(clean_title)
  )

cat("\nFinal cleaned data dimensions:\n")
```

```
##
## Final cleaned data dimensions:
```

```r
print(dim(car_model))
```

```
## [1] 3928   12
```

```r
cat("\nFinal numeric summaries:\n")
```

```
##
## Final numeric summaries:
```

```r
print(summary(car_model %>% select(price, log_price, age, mileage, log_mileage)))
```

```
##      price          log_price           age            mileage
##  Min.   :  2000   Min.   : 7.601   Min.   : 0.000   Min.   :   100
##  1st Qu.: 17000   1st Qu.: 9.741   1st Qu.: 4.000   1st Qu.: 24263
##  Median : 30500   Median :10.325   Median : 7.000   Median : 54000
##  Mean   : 37896   Mean   :10.258   Mean   : 8.526   Mean   : 65458
##  3rd Qu.: 48000   3rd Qu.:10.779   3rd Qu.:12.000   3rd Qu.: 95000
##  Max.   :200000   Max.   :12.206   Max.   :32.000   Max.   :300000
##   log_mileage
##  Min.   : 4.615
##  1st Qu.:10.097
##  Median :10.897
##  Mean   :10.646
##  3rd Qu.:11.462
##  Max.   :12.612
```

```r
cat("\nFinal categorical level counts:\n")
```

```
##
## Final categorical level counts:
```

```r
cat("brand_group levels:", nlevels(car_model$brand_group), "\n")
```

```
## brand_group levels: 35
```

```r
cat("fuel_type levels:", nlevels(car_model$fuel_type), "\n")
```

```
## fuel_type levels: 6
```

```r
cat("transmission levels:", nlevels(car_model$transmission), "\n")
```

```
## transmission levels: 5
```

```r
cat("accident levels:", nlevels(car_model$accident), "\n")
```

```
## accident levels: 3
```

```r
cat("clean_title levels:", nlevels(car_model$clean_title), "\n")
```

```
## clean_title levels: 2
```

```r
level_summary <- tibble(
  variable = c("brand_group", "fuel_type", "transmission", "accident", "clean_title"),
  n_levels = c(
    nlevels(car_model$brand_group),
    nlevels(car_model$fuel_type),
    nlevels(car_model$transmission),
    nlevels(car_model$accident),
    nlevels(car_model$clean_title)
  )
```

```
)

cat("\nLevel summary table:\n")
```

```
##
## Level summary table:
```

```
print(level_summary)
```

```
## # A tibble: 5 x 2
##   variable     n_levels
##   <chr>           <int>
## 1 brand_group        35
## 2 fuel_type           6
## 3 transmission        5
## 4 accident            3
## 5 clean_title         2
```

```
# 8. Train/test split

set.seed(431)

car_model <- car_model %>%
  mutate(row_id = row_number())

train_ids <- car_model %>%
  group_by(brand_group) %>%
  sample_frac(size = 0.80) %>%
  ungroup() %>%
  pull(row_id)

train_data <- car_model %>%
  filter(row_id %in% train_ids)

test_data <- car_model %>%
  filter(!(row_id %in% train_ids))

cat("\nTraining and testing sample sizes:\n")
```

```
##
## Training and testing sample sizes:
```

```
cat("Train n =", nrow(train_data), "\n")
```

```
## Train n = 3143
```

```
cat("Test n  =", nrow(test_data), "\n")
```

```
## Test n  = 785
```

```
# 9. Standardize continuous variables
# Important: compute mean/sd from training data only.

age_mean <- mean(train_data$age)
age_sd <- sd(train_data$age)
log_mileage_mean <- mean(train_data$log_mileage)
log_mileage_sd <- sd(train_data$log_mileage)
```

```r
y_mean <- mean(train_data$log_price)

train_data <- train_data %>%
  mutate(
    z_age = (age - age_mean) / age_sd,
    z_log_mileage = (log_mileage - log_mileage_mean) / log_mileage_sd,
    y = log_price - y_mean
  )

test_data <- test_data %>%
  mutate(
    z_age = (age - age_mean) / age_sd,
    z_log_mileage = (log_mileage - log_mileage_mean) / log_mileage_sd,
    y = log_price - y_mean
  )

scaling_info <- tibble(
  parameter = c("age_mean", "age_sd", "log_mileage_mean", "log_mileage_sd", "y_mean"),
  value = c(age_mean, age_sd, log_mileage_mean, log_mileage_sd, y_mean)
)

cat("\nScaling constants:\n")
```

```
##
## Scaling constants:
```

```r
print(scaling_info)
```

```
## # A tibble: 5 x 2
##   parameter        value
##   <chr>            <dbl>
## 1 age_mean          8.53
## 2 age_sd            6.06
## 3 log_mileage_mean 10.7
## 4 log_mileage_sd    1.13
## 5 y_mean           10.3
```

```r
# 10. Prepare model matrix and brand indices for JAGS

brand_levels <- levels(train_data$brand_group)
J <- length(brand_levels)

train_data <- train_data %>%
  mutate(brand_id = as.integer(factor(brand_group, levels = brand_levels)))

test_data <- test_data %>%
  mutate(brand_id = as.integer(factor(brand_group, levels = brand_levels)))

x_formula <- ~ z_age + z_log_mileage + fuel_type + transmission + accident + clean_title

X_train <- model.matrix(x_formula, data = train_data)
X_train <- X_train[, colnames(X_train) != "(Intercept)", drop = FALSE]

X_test_raw <- model.matrix(x_formula, data = test_data)
X_test_raw <- X_test_raw[, colnames(X_test_raw) != "(Intercept)", drop = FALSE]
```

```r
align_matrix <- function(X, target_colnames) {
  X_aligned <- matrix(0, nrow = nrow(X), ncol = length(target_colnames))
  colnames(X_aligned) <- target_colnames
  common_cols <- intersect(colnames(X), target_colnames)
  X_aligned[, common_cols] <- X[, common_cols, drop = FALSE]
  return(X_aligned)
}

X_test <- align_matrix(X_test_raw, colnames(X_train))

jags_data <- list(
  n = nrow(train_data),
  p = ncol(X_train),
  J = J,
  y = train_data$y,
  X = X_train,
  brand_id = train_data$brand_id
)

cat("\nJAGS data dimensions:\n")
```

```
##
## JAGS data dimensions:
```

```r
cat("n =", jags_data$n, "\n")
```

```
## n = 3143
```

```r
cat("p =", jags_data$p, "\n")
```

```
## p = 14
```

```r
cat("J =", jags_data$J, "\n")
```

```
## J = 35
```

```r
# 11. Save cleaned outputs

write_csv(car_model, p("data/processed/used_cars_cleaned_all.csv"))
write_csv(train_data, p("data/processed/used_cars_train_cleaned.csv"))
write_csv(test_data, p("data/processed/used_cars_test_cleaned.csv"))
write_csv(level_summary, p("data/processed/used_cars_level_summary.csv"))
write_csv(scaling_info, p("data/processed/used_cars_scaling_info.csv"))

saveRDS(
  list(
    train_data = train_data,
    test_data = test_data,
    X_train = X_train,
    X_test = X_test,
    jags_data = jags_data,
    brand_levels = brand_levels,
    scaling_info = scaling_info,
    x_formula = x_formula
  ),
  p("data/processed/used_cars_model_inputs.rds")
```

```
)

cat("\nSaved files:\n")
```

```
##
## Saved files:
```

```
cat("- used_cars_cleaned_all.csv\n")
```

```
## - used_cars_cleaned_all.csv
```

```
cat("- used_cars_train_cleaned.csv\n")
```

```
## - used_cars_train_cleaned.csv
```

```
cat("- used_cars_test_cleaned.csv\n")
```

```
## - used_cars_test_cleaned.csv
```

```
cat("- used_cars_level_summary.csv\n")
```

```
## - used_cars_level_summary.csv
```

```
cat("- used_cars_scaling_info.csv\n")
```

```
## - used_cars_scaling_info.csv
```

```
cat("- used_cars_model_inputs.rds\n")
```

```
## - used_cars_model_inputs.rds
```

```
# 12. EDA summary figure (Section 2)

library(patchwork)

top_brand_groups <- car_model %>%
  count(brand_group, sort = TRUE) %>%
  slice_head(n = 10) %>%
  pull(brand_group)

eda_brands <- car_model %>%
  filter(brand_group %in% c(top_brand_groups, "Other")) %>%
  count(brand_group, sort = TRUE) %>%
  mutate(brand_group = fct_reorder(brand_group, n, .desc = TRUE))

p_log_price <- ggplot(car_model, aes(x = log_price)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "white", alpha = 0.85) +
  labs(
    x = "log(price)",
    y = "Count",
    title = "A. Listing price (log scale)"
  ) +
  theme_minimal(base_size = 11)

eda_predictors <- car_model %>%
  select(age, log_mileage) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "value") %>%
  mutate(
    variable = recode(
```

```
      variable,
      age = "Age (years)",
      log_mileage = "log(mileage + 1)"
    )
  )

p_age_mileage <- ggplot(eda_predictors, aes(x = value)) +
  geom_histogram(bins = 30, fill = "gray50", color = "white", alpha = 0.85) +
  facet_wrap(~variable, scales = "free_x", nrow = 1) +
  labs(x = NULL, y = "Count", title = "B. Age and mileage") +
  theme_minimal(base_size = 11) +
  theme(strip.text = element_text(face = "bold"))

p_brand_counts <- ggplot(eda_brands, aes(x = n, y = brand_group)) +
  geom_col(fill = "gray30") +
  labs(
    x = "Listings",
    y = NULL,
    title = "C. Top brand groups (plus Other)"
  ) +
  theme_minimal(base_size = 11)

eda_figure <- (p_log_price | p_age_mileage) / p_brand_counts +
  plot_layout(heights = c(1, 1.05), widths = c(0.95, 1.25))

ggsave(
  p("results/eda/used_cars_eda_summary.pdf"),
  eda_figure,
  width = 11,
  height = 8
)
ggsave(
  p("results/eda/used_cars_eda_summary.png"),
  eda_figure,
  width = 11,
  height = 8,
  dpi = 300
)

cat("\nSaved EDA figure:\n")
```

```
##
## Saved EDA figure:
```

```
cat("- used_cars_eda_summary.pdf\n")
```

```
## - used_cars_eda_summary.pdf
```

```
cat("- used_cars_eda_summary.png\n")
```

```
## - used_cars_eda_summary.png
```

```
# 13. Final checks

cat("\nFinal checks:\n")
```

```
##
## Final checks:
cat("Any missing values in final model data? ", any(is.na(car_model)), "\n")
```

```
## Any missing values in final model data?  FALSE
cat("Any missing values in training data? ", any(is.na(train_data)), "\n")
```

```
## Any missing values in training data?  FALSE
cat("Any missing values in testing data? ", any(is.na(test_data)), "\n")
```

```
## Any missing values in testing data?  FALSE
cat("All test brands seen in training? ", all(test_data$brand_group %in% train_data$brand_group), "\n")
```

```
## All test brands seen in training?  TRUE
cat("Number of fixed-effect columns in X_train:", ncol(X_train), "\n")
```

```
## Number of fixed-effect columns in X_train: 14
cat("Number of fixed-effect columns in X_test:", ncol(X_test), "\n")
```

```
## Number of fixed-effect columns in X_test: 14
# Bayesian Hierarchical Model for Used Car Price Prediction
# Step 2: Fit model using JAGS


library(tidyverse)
library(rjags)
```

```
## Loading required package: coda
```

```
## Linked to JAGS 4.3.2
```

```
## Loaded modules: basemod,bugs
library(coda)

# 1. Load cleaned model inputs

inputs <- readRDS(p("data/processed/used_cars_model_inputs.rds"))

train_data <- inputs$train_data
test_data <- inputs$test_data
X_train <- inputs$X_train
X_test <- inputs$X_test
jags_data <- inputs$jags_data
brand_levels <- inputs$brand_levels
scaling_info <- inputs$scaling_info

cat("Training observations:", nrow(train_data), "\n")
```

```
## Training observations: 3143
cat("Testing observations:", nrow(test_data), "\n")
```

```
## Testing observations: 785
```

```r
cat("Number of fixed-effect predictors p:", jags_data$p, "\n")
```

```
## Number of fixed-effect predictors p: 14
```

```r
cat("Number of brand groups J:", jags_data$J, "\n")
```

```
## Number of brand groups J: 35
```

```r
# 2. Write JAGS model file

model_string <- "
model {

  # ----------------------------
  # Likelihood
  # ----------------------------
  for (i in 1:n) {
    y[i] ~ dnorm(mu[i], tau)
    mu[i] <- alpha[brand_id[i]] + inprod(X[i,], beta[])
  }

  # ----------------------------
  # Fixed-effect priors
  # ----------------------------
  for (k in 1:p) {
    beta[k] ~ dnorm(0, 0.25)
  }

  # ----------------------------
  # Brand-level random intercepts
  # ----------------------------
  for (j in 1:J) {
    alpha[j] ~ dnorm(mu_alpha, tau_alpha)
  }

  # ----------------------------
  # Hyperpriors
  # ----------------------------
  mu_alpha ~ dnorm(0, 0.04)

  sigma ~ dnorm(0, 0.25) T(0,)
  sigma_alpha ~ dnorm(0, 0.25) T(0,)

  tau <- 1 / (sigma * sigma)
  tau_alpha <- 1 / (sigma_alpha * sigma_alpha)
}
"

writeLines(model_string, con = p("models/jags/used_car_hier_model.txt"))

# 3. Initial values

set.seed(431)

init_function <- function() {
```

```
  list(
    beta = rnorm(jags_data$p, 0, 0.1),
    alpha = rnorm(jags_data$J, 0, 0.1),
    mu_alpha = rnorm(1, 0, 0.1),
    sigma = runif(1, 0.5, 1.5),
    sigma_alpha = runif(1, 0.2, 1.0)
  )
}

# 4. Create and adapt JAGS model

n_chains <- 3

jags_model <- jags.model(
  file = p("models/jags/used_car_hier_model.txt"),
  data = jags_data,
  inits = init_function,
  n.chains = n_chains,
  n.adapt = 1000
)
```

```
## Compiling model graph
##    Resolving undeclared variables
##    Allocating nodes
## Graph information:
##    Observed stochastic nodes: 3143
##    Unobserved stochastic nodes: 52
##    Total graph size: 59714
##
## Initializing model
```

```
# 5. Burn-in

update(jags_model, n.iter = 5000)

# 6. Draw posterior samples

params_to_monitor <- c(
  "beta",
  "alpha",
  "mu_alpha",
  "sigma",
  "sigma_alpha"
)

samples <- coda.samples(
  model = jags_model,
  variable.names = params_to_monitor,
  n.iter = 10000,
  thin = 5
)

# 6b. DIC (requires compiled jags_model in memory)
dic_baseline <- dic.samples(jags_model, n.iter = 2000, thin = 5)
```

```r
cat("\nDIC (baseline hierarchical model, training data):\n")
```

```
##
## DIC (baseline hierarchical model, training data):
```

```r
print(dic_baseline)
```

```
## Mean deviance:  3711
## penalty 48.73
## Penalized deviance: 3760
```

```r
saveRDS(dic_baseline, p("results/mcmc/used_car_dic_baseline.rds"))

# 7. Posterior summary

summary_samples <- summary(samples)

cat("\nPosterior summary:\n")
```

```
##
## Posterior summary:
```

```r
print(summary_samples)
```

```
##
## Iterations = 6005:16000
## Thinning interval = 5
## Number of chains = 3
## Sample size per chain = 2000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##                 Mean       SD  Naive SE Time-series SE
## alpha[1]     0.105926 0.082314 1.063e-03      3.066e-03
## alpha[2]     0.395593 0.066810 8.625e-04      3.056e-03
## alpha[3]     1.417263 0.111813 1.443e-03      3.100e-03
## alpha[4]     0.447865 0.064880 8.376e-04      3.149e-03
## alpha[5]     0.003569 0.103811 1.340e-03      3.067e-03
## alpha[6]     0.407435 0.074533 9.622e-04      3.042e-03
## alpha[7]     0.400800 0.064331 8.305e-04      3.032e-03
## alpha[8]    -0.144660 0.107886 1.393e-03      3.014e-03
## alpha[9]     0.252980 0.076496 9.876e-04      2.940e-03
## alpha[10]    0.316374 0.060037 7.751e-04      2.851e-03
## alpha[11]    0.148948 0.118066 1.524e-03      2.895e-03
## alpha[12]    0.412741 0.074815 9.659e-04      2.833e-03
## alpha[13]    0.249676 0.084523 1.091e-03      3.087e-03
## alpha[14]   -0.204705 0.081798 1.056e-03      3.237e-03
## alpha[15]    0.211109 0.085220 1.100e-03      3.004e-03
## alpha[16]    0.483191 0.090281 1.166e-03      2.982e-03
## alpha[17]    0.279299 0.070814 9.142e-04      2.987e-03
## alpha[18]   -0.066926 0.079697 1.029e-03      3.066e-03
## alpha[19]    0.601065 0.073000 9.424e-04      3.206e-03
## alpha[20]    0.436595 0.069357 8.954e-04      3.060e-03
## alpha[21]    0.275266 0.087973 1.136e-03      3.004e-03
## alpha[22]    0.463906 0.105782 1.366e-03      3.117e-03
```

```
## alpha[23]   -0.068222 0.085704 1.106e-03      3.169e-03
## alpha[24]    0.576712 0.062929 8.124e-04      3.012e-03
## alpha[25]   -0.262884 0.103064 1.331e-03      3.286e-03
## alpha[26]    0.142651 0.121719 1.571e-03      3.153e-03
## alpha[27]    0.129132 0.074839 9.662e-04      3.221e-03
## alpha[28]    0.544128 0.074308 9.593e-04      3.141e-03
## alpha[29]    0.994677 0.069314 8.948e-04      2.984e-03
## alpha[30]    0.431996 0.070798 9.140e-04      2.462e-03
## alpha[31]    0.144139 0.088957 1.148e-03      3.244e-03
## alpha[32]    0.519886 0.089925 1.161e-03      3.168e-03
## alpha[33]    0.290264 0.068488 8.842e-04      3.077e-03
## alpha[34]   -0.091522 0.087967 1.136e-03      3.114e-03
## alpha[35]    0.107507 0.096376 1.244e-03      3.098e-03
## beta[1]     -0.363125 0.010873 1.404e-04      1.433e-04
## beta[2]     -0.282171 0.010518 1.358e-04      1.447e-04
## beta[3]     -0.542363 0.063379 8.182e-04      2.536e-03
## beta[4]     -0.463639 0.051071 6.593e-04      2.617e-03
## beta[5]     -0.483981 0.063323 8.175e-04      2.633e-03
## beta[6]     -0.501116 0.104535 1.350e-03      2.831e-03
## beta[7]     -0.550102 0.067113 8.664e-04      2.633e-03
## beta[8]     -0.244976 0.054338 7.015e-04      7.015e-04
## beta[9]      0.161149 0.029913 3.862e-04      3.920e-04
## beta[10]    -0.070013 0.111496 1.439e-03      1.440e-03
## beta[11]     0.423595 0.302716 3.908e-03      3.972e-03
## beta[12]     0.091473 0.018684 2.412e-04      3.264e-04
## beta[13]     0.127091 0.057360 7.405e-04      1.060e-03
## beta[14]     0.009648 0.026358 3.403e-04      6.769e-04
## mu_alpha     0.295927 0.082373 1.063e-03      3.012e-03
## sigma        0.436830 0.005582 7.207e-05      7.137e-05
## sigma_alpha  0.345054 0.046686 6.027e-04      6.028e-04
##
## 2. Quantiles for each variable:
##
##                2.5%       25%       50%       75%    97.5%
## alpha[1]    -0.05813  0.051244  0.107632  0.162213  0.26192
## alpha[2]     0.26529  0.351181  0.396433  0.439686  0.52829
## alpha[3]     1.19658  1.342249  1.417592  1.490788  1.63925
## alpha[4]     0.31874  0.404659  0.448074  0.490862  0.57728
## alpha[5]    -0.19525 -0.067424  0.002145  0.072595  0.20881
## alpha[6]     0.26334  0.357349  0.406691  0.457675  0.55278
## alpha[7]     0.26951  0.357792  0.402103  0.442309  0.52813
## alpha[8]    -0.35710 -0.215958 -0.145080 -0.070658  0.06785
## alpha[9]     0.10479  0.201880  0.252518  0.304378  0.40157
## alpha[10]    0.19875  0.276894  0.316804  0.356620  0.43308
## alpha[11]   -0.08110  0.068624  0.149982  0.227674  0.37719
## alpha[12]    0.26269  0.362912  0.412925  0.463175  0.56015
## alpha[13]    0.08128  0.194909  0.251659  0.306298  0.41112
## alpha[14]   -0.36587 -0.261092 -0.204771 -0.149450 -0.04484
## alpha[15]    0.04067  0.152923  0.212874  0.268332  0.37690
## alpha[16]    0.30741  0.421204  0.484193  0.544756  0.65996
## alpha[17]    0.14252  0.230900  0.278931  0.325634  0.41931
## alpha[18]   -0.22465 -0.120222 -0.066468 -0.014300  0.08999
## alpha[19]    0.45471  0.551719  0.600836  0.649941  0.74435
## alpha[20]    0.30338  0.390433  0.437172  0.483132  0.57623
```

```
## alpha[21]    0.10087  0.216500  0.275730  0.333907  0.44767
## alpha[22]    0.25355  0.394122  0.464779  0.533858  0.67299
## alpha[23]   -0.23672 -0.124723 -0.067700 -0.011015  0.10457
## alpha[24]    0.44963  0.534857  0.577403  0.617332  0.70274
## alpha[25]   -0.46366 -0.332277 -0.264173 -0.193931 -0.05618
## alpha[26]   -0.09912  0.059449  0.143462  0.222462  0.38306
## alpha[27]   -0.01949  0.078849  0.129152  0.179079  0.27461
## alpha[28]    0.39620  0.495150  0.544672  0.594430  0.69122
## alpha[29]    0.85916  0.949005  0.995416  1.039155  1.13431
## alpha[30]    0.29335  0.383922  0.432201  0.479827  0.57059
## alpha[31]   -0.03316  0.083708  0.144065  0.205236  0.31756
## alpha[32]    0.34047  0.460520  0.520586  0.579799  0.69743
## alpha[33]    0.15617  0.243615  0.290793  0.336149  0.42483
## alpha[34]   -0.26420 -0.149735 -0.091037 -0.032528  0.08049
## alpha[35]   -0.08216  0.042253  0.106825  0.173076  0.29371
## beta[1]     -0.38474 -0.370346 -0.363237 -0.355663 -0.34210
## beta[2]     -0.30297 -0.289226 -0.282154 -0.275028 -0.26182
## beta[3]     -0.66639 -0.584875 -0.542578 -0.500522 -0.41738
## beta[4]     -0.56593 -0.496705 -0.464233 -0.430245 -0.36271
## beta[5]     -0.61179 -0.526892 -0.484006 -0.442746 -0.36010
## beta[6]     -0.70588 -0.569001 -0.501739 -0.431958 -0.29183
## beta[7]     -0.68175 -0.593888 -0.550240 -0.505632 -0.41755
## beta[8]     -0.35384 -0.281020 -0.244927 -0.208864 -0.13852
## beta[9]      0.10226  0.141051  0.160679  0.181920  0.21952
## beta[10]    -0.29168 -0.143840 -0.070691  0.004849  0.14906
## beta[11]    -0.16983  0.222264  0.422577  0.624157  1.01980
## beta[12]     0.05541  0.078553  0.091578  0.104240  0.12807
## beta[13]     0.01495  0.088525  0.127252  0.165661  0.23983
## beta[14]    -0.04015 -0.008128  0.009313  0.027456  0.06263
## mu_alpha     0.13472  0.240680  0.295396  0.350227  0.45703
## sigma        0.42592  0.433044  0.436799  0.440614  0.44770
## sigma_alpha  0.26557  0.312671  0.339837  0.372858  0.44998
```

```
# 8. Convergence diagnostics

cat("\nGelman-Rubin diagnostics:\n")
```

```
##
## Gelman-Rubin diagnostics:
```

```
gelman_results <- gelman.diag(samples, multivariate = FALSE)
print(gelman_results)
```

```
## Potential scale reduction factors:
##
##             Point est. Upper C.I.
## alpha[1]             1       1.01
## alpha[2]             1       1.01
## alpha[3]             1       1.00
## alpha[4]             1       1.01
## alpha[5]             1       1.01
## alpha[6]             1       1.01
## alpha[7]             1       1.01
## alpha[8]             1       1.01
## alpha[9]             1       1.01
```

```
## alpha[10]            1       1.01
## alpha[11]            1       1.01
## alpha[12]            1       1.01
## alpha[13]            1       1.00
## alpha[14]            1       1.00
## alpha[15]            1       1.00
## alpha[16]            1       1.01
## alpha[17]            1       1.01
## alpha[18]            1       1.01
## alpha[19]            1       1.00
## alpha[20]            1       1.01
## alpha[21]            1       1.00
## alpha[22]            1       1.00
## alpha[23]            1       1.00
## alpha[24]            1       1.01
## alpha[25]            1       1.00
## alpha[26]            1       1.00
## alpha[27]            1       1.01
## alpha[28]            1       1.01
## alpha[29]            1       1.01
## alpha[30]            1       1.00
## alpha[31]            1       1.00
## alpha[32]            1       1.01
## alpha[33]            1       1.00
## alpha[34]            1       1.00
## alpha[35]            1       1.00
## beta[1]              1       1.00
## beta[2]              1       1.00
## beta[3]              1       1.01
## beta[4]              1       1.01
## beta[5]              1       1.00
## beta[6]              1       1.00
## beta[7]              1       1.00
## beta[8]              1       1.00
## beta[9]              1       1.00
## beta[10]             1       1.00
## beta[11]             1       1.00
## beta[12]             1       1.00
## beta[13]             1       1.01
## beta[14]             1       1.00
## mu_alpha             1       1.00
## sigma                1       1.00
## sigma_alpha          1       1.00
```

```
cat("\nEffective sample sizes:\n")
```

```
##
## Effective sample sizes:
```

```
ess_results <- effectiveSize(samples)
print(ess_results)
```

```
##    alpha[1]    alpha[2]    alpha[3]    alpha[4]    alpha[5]    alpha[6]
##    727.7033    490.4029   1302.1154    430.7579   1150.0296    609.0067
##    alpha[7]    alpha[8]    alpha[9]   alpha[10]   alpha[11]   alpha[12]
```

```
##    452.9828   1318.5390    680.8388    444.1741   1715.1486    696.9367
##   alpha[13]   alpha[14]   alpha[15]   alpha[16]   alpha[17]   alpha[18]
##    754.7800    639.8254    823.7328    915.7189    572.8703    679.0410
##   alpha[19]   alpha[20]   alpha[21]   alpha[22]   alpha[23]   alpha[24]
##    520.3967    515.5533    866.4614   1157.7675    740.4595    437.3141
##   alpha[25]   alpha[26]   alpha[27]   alpha[28]   alpha[29]   alpha[30]
##    984.3872   1526.0878    545.5019    562.5487    544.2844    836.6011
##   alpha[31]   alpha[32]   alpha[33]   alpha[34]   alpha[35]     beta[1]
##    778.3447    809.7360    498.3450    806.6096   1005.9430   5766.8298
##     beta[2]     beta[3]     beta[4]     beta[5]     beta[6]     beta[7]
##   5302.1641    631.8628    380.4312    580.3130   1363.1127    650.4844
##     beta[8]     beta[9]    beta[10]    beta[11]    beta[12]    beta[13]
##   6000.0000   5829.9879   6000.0000   5816.5609   3311.9182   2928.3224
##    beta[14]    mu_alpha       sigma sigma_alpha
##   1517.5017    767.4715   6123.5794   6000.0000
```

```
# 8b. Monte Carlo standard errors (baseline model)
# Key estimands: residual scale (sigma), brand-level spread (sigma_alpha),
# overall brand intercept (mu_alpha), and first fixed-effect betas.
mcse_targets <- c(
  "sigma",
  "sigma_alpha",
  "mu_alpha",
  paste0("beta[", seq_len(min(3, jags_data$p)), "]")
)

sum_stats <- summary(samples)$statistics
mcse_results <- sum_stats[mcse_targets, "Time-series SE", drop = TRUE]
names(mcse_results) <- mcse_targets

cat("\nMonte Carlo standard errors (time-series SE from coda::summary):\n")
```

```
##
## Monte Carlo standard errors (time-series SE from coda::summary):
```

```
print(mcse_results)
```

```
##        sigma  sigma_alpha     mu_alpha      beta[1]      beta[2]      beta[3]
## 0.0000713743 0.0006028027 0.0030121610 0.0001432907 0.0001447120 0.0025363801
```

```
mcse_summary <- tibble(
  parameter = mcse_targets,
  posterior_mean = sum_stats[mcse_targets, "Mean"],
  posterior_sd = sum_stats[mcse_targets, "SD"],
  ess = as.numeric(ess_results[mcse_targets]),
  mcse = as.numeric(mcse_results),
) %>%
  mutate(mcse_over_sd = mcse / posterior_sd)

cat("\nMCSE summary table:\n")
```

```
##
## MCSE summary table:
```

```
print(mcse_summary)
```

```
## # A tibble: 6 x 6
```

```
##   parameter   posterior_mean posterior_sd   ess      mcse mcse_over_sd
##   <chr>                <dbl>        <dbl> <dbl>     <dbl>        <dbl>
## 1 sigma                0.437      0.00558 6124. 0.0000714       0.0128
## 2 sigma_alpha          0.345      0.0467  6000  0.000603        0.0129
## 3 mu_alpha             0.296      0.0824   767. 0.00301         0.0366
## 4 beta[1]             -0.363      0.0109  5767. 0.000143        0.0132
## 5 beta[2]             -0.282      0.0105  5302. 0.000145        0.0138
## 6 beta[3]             -0.542      0.0634   632. 0.00254         0.0400
```

```
write_csv(mcse_summary, p("results/mcmc/mcmc_mcse_summary.csv"))

# 9. Trace plots

pdf(p("results/mcmc/traceplots_main_parameters.pdf"), width = 10, height = 8)
traceplot(samples[, c("mu_alpha", "sigma", "sigma_alpha")])
dev.off()
```

```
## pdf
##   2
```

```
beta_names <- paste0("beta[", 1:min(6, jags_data$p), "]")

pdf(p("results/mcmc/traceplots_beta_parameters.pdf"), width = 10, height = 8)
traceplot(samples[, beta_names])
dev.off()
```

```
## pdf
##   2
```

```
# 10. Save model outputs

saveRDS(samples, p("results/mcmc/used_car_jags_samples.rds"))
saveRDS(gelman_results, p("results/mcmc/used_car_gelman_results.rds"))
saveRDS(ess_results, p("results/mcmc/used_car_effective_sample_sizes.rds"))
saveRDS(summary_samples, p("results/mcmc/used_car_posterior_summary.rds"))
saveRDS(mcse_results, p("results/mcmc/used_car_mcse_results.rds"))

cat("\nSaved files:\n")
```

```
##
## Saved files:
```

```
cat("- used_car_hier_model.txt\n")
```

```
## - used_car_hier_model.txt
```

```
cat("- used_car_jags_samples.rds\n")
```

```
## - used_car_jags_samples.rds
```

```
cat("- used_car_dic_baseline.rds\n")
```

```
## - used_car_dic_baseline.rds
```

```
cat("- used_car_gelman_results.rds\n")
```

```
## - used_car_gelman_results.rds
```

```
cat("- used_car_effective_sample_sizes.rds\n")
```

```
## - used_car_effective_sample_sizes.rds
cat("- used_car_mcse_results.rds\n")

## - used_car_mcse_results.rds
cat("- mcmc_mcse_summary.csv\n")

## - mcmc_mcse_summary.csv
cat("- used_car_posterior_summary.rds\n")

## - used_car_posterior_summary.rds
cat("- traceplots_main_parameters.pdf\n")

## - traceplots_main_parameters.pdf
cat("- traceplots_beta_parameters.pdf\n")

## - traceplots_beta_parameters.pdf
# Prior sensitivity: alternative fit (tighter beta prior only)

library(tidyverse)
library(rjags)
library(coda)

inputs_ps <- readRDS(p("data/processed/used_cars_model_inputs.rds"))
jags_data_ps <- inputs_ps$jags_data

model_string_alt <- "
model {
  for (i in 1:n) {
    y[i] ~ dnorm(mu[i], tau)
    mu[i] <- alpha[brand_id[i]] + inprod(X[i,], beta[])
  }
  for (k in 1:p) {
    beta[k] ~ dnorm(0, 1)
  }
  for (j in 1:J) {
    alpha[j] ~ dnorm(mu_alpha, tau_alpha)
  }
  mu_alpha ~ dnorm(0, 0.04)
  sigma ~ dnorm(0, 0.25) T(0,)
  sigma_alpha ~ dnorm(0, 0.25) T(0,)
  tau <- 1 / (sigma * sigma)
  tau_alpha <- 1 / (sigma_alpha * sigma_alpha)
}
"

writeLines(model_string_alt, con = p("models/jags/used_car_hier_model_prior_alt.txt"))

set.seed(431)

init_function_alt <- function() {
  list(
    beta = rnorm(jags_data_ps$p, 0, 0.1),
```

```
    alpha = rnorm(jags_data_ps$J, 0, 0.1),
    mu_alpha = rnorm(1, 0, 0.1),
    sigma = runif(1, 0.5, 1.5),
    sigma_alpha = runif(1, 0.2, 1.0)
  )
}

jags_model_alt <- jags.model(
  file = p("models/jags/used_car_hier_model_prior_alt.txt"),
  data = jags_data_ps,
  inits = init_function_alt,
  n.chains = 3,
  n.adapt = 1000
)
```

```
## Compiling model graph
##    Resolving undeclared variables
##    Allocating nodes
## Graph information:
##    Observed stochastic nodes: 3143
##    Unobserved stochastic nodes: 52
##    Total graph size: 59714
##
## Initializing model
```

```
update(jags_model_alt, n.iter = 5000)

samples_alt <- coda.samples(
  model = jags_model_alt,
  variable.names = c("beta", "alpha", "mu_alpha", "sigma", "sigma_alpha"),
  n.iter = 10000,
  thin = 5
)

dic_alt <- dic.samples(jags_model_alt, n.iter = 2000, thin = 5)
cat("\nDIC (alternative beta prior, training data):\n")
```

```
##
## DIC (alternative beta prior, training data):
```

```
print(dic_alt)
```

```
## Mean deviance:  3711
## penalty 48.87
## Penalized deviance: 3760
```

```
saveRDS(dic_alt, p("results/prior_sensitivity/used_car_dic_prior_alt.rds"))

saveRDS(samples_alt, p("results/prior_sensitivity/used_car_jags_samples_prior_alt.rds"))

m_base <- as.matrix(readRDS(p("results/mcmc/used_car_jags_samples.rds")))
m_alt <- as.matrix(samples_alt)

summ_par <- function(mat, par) {
  c(mean = mean(mat[, par]), q025 = quantile(mat[, par], 0.025), q975 = quantile(mat[, par], 0.975))
}
```

```r
prior_sens_compare <- tibble::tibble(
  parameter = c("sigma", "sigma_alpha", "mu_alpha", "beta[1]", "beta[2]", "beta[3]"),
  baseline_mean = c(
    mean(m_base[, "sigma"]),
    mean(m_base[, "sigma_alpha"]),
    mean(m_base[, "mu_alpha"]),
    mean(m_base[, "beta[1]"]),
    mean(m_base[, "beta[2]"]),
    mean(m_base[, "beta[3]"])
  ),
  alt_mean = c(
    mean(m_alt[, "sigma"]),
    mean(m_alt[, "sigma_alpha"]),
    mean(m_alt[, "mu_alpha"]),
    mean(m_alt[, "beta[1]"]),
    mean(m_alt[, "beta[2]"]),
    mean(m_alt[, "beta[3]"])
  )
) %>%
  mutate(diff = alt_mean - baseline_mean)

cat("\nPrior sensitivity: posterior mean comparison (alt - baseline beta prior)\n")
```

```
##
## Prior sensitivity: posterior mean comparison (alt - baseline beta prior)
```

```r
print(prior_sens_compare)
```

```
## # A tibble: 6 x 4
##   parameter   baseline_mean alt_mean        diff
##   <chr>               <dbl>    <dbl>       <dbl>
## 1 sigma               0.437    0.437  0.00000507
## 2 sigma_alpha         0.345    0.346  0.000999
## 3 mu_alpha            0.296    0.294 -0.00149
## 4 beta[1]            -0.363   -0.363  0.000324
## 5 beta[2]            -0.282   -0.282 -0.0000758
## 6 beta[3]            -0.542   -0.538  0.00414
```

```r
write_csv(prior_sens_compare, p("results/prior_sensitivity/prior_sensitivity_posterior_compare.csv"))

test_data_ps <- inputs_ps$test_data
X_test_ps <- inputs_ps$X_test
scaling_info_ps <- inputs_ps$scaling_info
brand_levels_ps <- inputs_ps$brand_levels
y_mean_ps <- scaling_info_ps$value[scaling_info_ps$parameter == "y_mean"]

p_ps <- ncol(X_test_ps)
J_ps <- length(brand_levels_ps)
beta_cols_ps <- paste0("beta[", 1:p_ps, "]")
alpha_cols_ps <- paste0("alpha[", 1:J_ps, "]")

beta_alt <- m_alt[, beta_cols_ps, drop = FALSE]
alpha_alt <- m_alt[, alpha_cols_ps, drop = FALSE]
sigma_alt <- m_alt[, "sigma"]
```

```r
n_test_ps <- nrow(test_data_ps)
n_draws_alt <- nrow(m_alt)
pred_log_alt <- matrix(NA, nrow = n_draws_alt, ncol = n_test_ps)

for (s in seq_len(n_draws_alt)) {
  fixed_part <- as.vector(X_test_ps %*% beta_alt[s, ])
  brand_part <- alpha_alt[s, test_data_ps$brand_id]
  mu_log_price <- fixed_part + brand_part + y_mean_ps
  pred_log_alt[s, ] <- rnorm(n_test_ps, mean = mu_log_price, sd = sigma_alt[s])
}

pred_price_alt <- exp(pred_log_alt)
bayes_pred_price_alt <- colMeans(pred_price_alt)
actual_ps <- test_data_ps$price
rmse_ps <- function(a, p) sqrt(mean((a - p)^2))
mae_ps <- function(a, p) mean(abs(a - p))

bayes_rmse_alt <- rmse_ps(actual_ps, bayes_pred_price_alt)
bayes_mae_alt <- mae_ps(actual_ps, bayes_pred_price_alt)

beta_base <- m_base[, beta_cols_ps, drop = FALSE]
alpha_base <- m_base[, alpha_cols_ps, drop = FALSE]
sigma_base_vec <- m_base[, "sigma"]
n_draws_base <- nrow(m_base)
pred_log_base <- matrix(NA, nrow = n_draws_base, ncol = n_test_ps)
for (s in seq_len(n_draws_base)) {
  fixed_part <- as.vector(X_test_ps %*% beta_base[s, ])
  brand_part <- alpha_base[s, test_data_ps$brand_id]
  mu_log_price <- fixed_part + brand_part + y_mean_ps
  pred_log_base[s, ] <- rnorm(n_test_ps, mean = mu_log_price, sd = sigma_base_vec[s])
}
bayes_pred_price_base <- colMeans(exp(pred_log_base))
bayes_rmse_base <- rmse_ps(actual_ps, bayes_pred_price_base)
bayes_mae_base <- mae_ps(actual_ps, bayes_pred_price_base)

pred_metric_compare <- tibble::tibble(
  prior = c("beta ~ N(0, 2^2) baseline", "beta ~ N(0, 1^2) alternative"),
  test_RMSE = c(bayes_rmse_base, bayes_rmse_alt),
  test_MAE = c(bayes_mae_base, bayes_mae_alt)
)

cat("\nPrior sensitivity: test-set prediction metrics\n")
```

```
##
## Prior sensitivity: test-set prediction metrics
```

```r
print(pred_metric_compare)
```

```
## # A tibble: 2 x 3
##   prior                          test_RMSE test_MAE
##   <chr>                              <dbl>    <dbl>
## 1 beta ~ N(0, 2^2) baseline       24075.   13431.
## 2 beta ~ N(0, 1^2) alternative    24024.   13428.
```

```r
write_csv(pred_metric_compare, p("results/prior_sensitivity/prior_sensitivity_test_metrics.csv"))
```

```r
# WAIC / LOO-CV from pointwise log-likelihood


library(tidyverse)
library(loo)
```

```
## Warning: package 'loo' was built under R version 4.4.3
```

```
## This is loo version 2.9.0
```

```
## - Online documentation and vignettes at mc-stan.org/loo
```

```
## - As of v2.0.0 loo defaults to 1 core but we recommend using as many as possible. Use the 'cores' ar
```

```r
inputs_ic <- readRDS(p("data/processed/used_cars_model_inputs.rds"))
jags_ic <- inputs_ic$jags_data
train_ic <- inputs_ic$train_data
Xtr <- as.matrix(inputs_ic$X_train)
y_tr <- jags_ic$y
brand_id <- train_ic$brand_id
p_ic <- jags_ic$p
J_ic <- jags_ic$J

build_log_lik <- function(m, y, X, brand_id, p, J) {
  S <- nrow(m)
  n <- length(y)
  beta_cols <- paste0("beta[", seq_len(p), "]")
  alpha_cols <- paste0("alpha[", seq_len(J), "]")
  beta_s <- m[, beta_cols, drop = FALSE]
  alpha_s <- m[, alpha_cols, drop = FALSE]
  sig <- m[, "sigma"]
  log_lik <- matrix(NA_real_, nrow = S, ncol = n)
  for (s in seq_len(S)) {
    mu <- alpha_s[s, brand_id] + as.numeric(X %*% beta_s[s, ])
    log_lik[s, ] <- stats::dnorm(y, mean = mu, sd = sig[s], log = TRUE)
  }
  log_lik
}

m_base <- as.matrix(readRDS(p("results/mcmc/used_car_jags_samples.rds")))
m_alt <- as.matrix(readRDS(p("results/prior_sensitivity/used_car_jags_samples_prior_alt.rds")))

log_lik_base <- build_log_lik(m_base, y_tr, Xtr, brand_id, p_ic, J_ic)
log_lik_alt <- build_log_lik(m_alt, y_tr, Xtr, brand_id, p_ic, J_ic)

waic_base <- waic(log_lik_base)
```

```
## Warning:
## 6 (0.2%) p_waic estimates greater than 0.4. We recommend trying loo instead.
```

```r
waic_alt <- waic(log_lik_alt)
```

```
## Warning:
## 6 (0.2%) p_waic estimates greater than 0.4. We recommend trying loo instead.
```

```r
loo_base <- loo(log_lik_base, save_psis = TRUE)
loo_alt <- loo(log_lik_alt, save_psis = TRUE)

cat("\nWAIC - baseline:\n")
```

```
##
## WAIC - baseline:
```

```r
print(waic_base)
```

```
##
## Computed from 6000 by 3143 log-likelihood matrix.
##
##           Estimate    SE
## elpd_waic  -1879.9  50.4
## p_waic        47.0   2.1
## waic        3759.9 100.7
##
## 6 (0.2%) p_waic estimates greater than 0.4. We recommend trying loo instead.
```

```r
cat("\nWAIC - alternative beta prior:\n")
```

```
##
## WAIC - alternative beta prior:
```

```r
print(waic_alt)
```

```
##
## Computed from 6000 by 3143 log-likelihood matrix.
##
##           Estimate    SE
## elpd_waic  -1879.8  50.3
## p_waic        46.9   2.0
## waic        3759.5 100.7
##
## 6 (0.2%) p_waic estimates greater than 0.4. We recommend trying loo instead.
```

```r
cat("\nLOO - baseline:\n")
```

```
##
## LOO - baseline:
```

```r
print(loo_base)
```

```
##
## Computed from 6000 by 3143 log-likelihood matrix.
##
##          Estimate    SE
## elpd_loo  -1880.1  50.4
## p_loo        47.1   2.1
## looic      3760.1 100.7
## ------
## MCSE of elpd_loo is 0.1.
## MCSE and ESS estimates assume independent draws (r_eff=1).
##
## All Pareto k estimates are good (k < 0.7).
## See help('pareto-k-diagnostic') for details.
```

```r
cat("\nLOO - alternative beta prior:\n")
```

```
##
## LOO - alternative beta prior:
```

```r
print(loo_alt)
```

```
##
## Computed from 6000 by 3143 log-likelihood matrix.
##
##          Estimate    SE
## elpd_loo  -1879.9  50.3
## p_loo        47.0   2.0
## looic      3759.7 100.7
## ------
## MCSE of elpd_loo is 0.1.
## MCSE and ESS estimates assume independent draws (r_eff=1).
##
## All Pareto k estimates are good (k < 0.7).
## See help('pareto-k-diagnostic') for details.
```

```r
loo_comp <- loo_compare(list(baseline = loo_base, alternative_beta_prior = loo_alt))
cat("\nloo_compare (sorted by elpd; first row is best; elpd_diff is vs. that row):\n")
```

```
##
## loo_compare (sorted by elpd; first row is best; elpd_diff is vs. that row):
```

```r
print(loo_comp)
```

```
##                         elpd_diff se_diff
## alternative_beta_prior  0.0       0.0
## baseline               -0.2       0.1
```

```r
saveRDS(loo_base, p("results/model_comparison/used_car_loo_baseline.rds"))
saveRDS(loo_alt, p("results/model_comparison/used_car_loo_prior_alt.rds"))
saveRDS(waic_base, p("results/model_comparison/used_car_waic_baseline.rds"))
saveRDS(waic_alt, p("results/model_comparison/used_car_waic_prior_alt.rds"))

dic_b <- readRDS(p("results/mcmc/used_car_dic_baseline.rds"))
dic_a <- readRDS(p("results/prior_sensitivity/used_car_dic_prior_alt.rds"))

dic_tbl <- tibble(
  model = c("baseline_beta_N02", "alternative_beta_N01"),
  mean_deviance = c(mean(as.matrix(dic_b$deviance)), mean(as.matrix(dic_a$deviance))),
  penalty_pD = c(mean(as.matrix(dic_b$penalty)), mean(as.matrix(dic_a$penalty))),
  DIC = mean_deviance + penalty_pD
)

waic_tbl <- tibble(
  model = c("baseline_beta_N02", "alternative_beta_N01"),
  waic = c(waic_base$estimates["waic", "Estimate"], waic_alt$estimates["waic", "Estimate"]),
  se = c(waic_base$estimates["waic", "SE"], waic_alt$estimates["waic", "SE"])
)

loo_tbl <- tibble(
  model = c("baseline_beta_N02", "alternative_beta_N01"),
```

```
  elpd_loo = c(loo_base$estimates["elpd_loo", "Estimate"], loo_alt$estimates["elpd_loo", "Estimate"]),
  se_elpd = c(loo_base$estimates["elpd_loo", "SE"], loo_alt$estimates["elpd_loo", "SE"]),
  looic = c(loo_base$estimates["looic", "Estimate"], loo_alt$estimates["looic", "Estimate"]),
  p_loo = c(loo_base$estimates["p_loo", "Estimate"], loo_alt$estimates["p_loo", "Estimate"])
)

loo_comp_df <- tibble::rownames_to_column(as.data.frame(as.matrix(loo_comp)), "model") %>%
  as_tibble()

write_csv(dic_tbl, p("results/model_comparison/model_comparison_dic.csv"))
write_csv(waic_tbl, p("results/model_comparison/model_comparison_waic.csv"))
write_csv(loo_tbl, p("results/model_comparison/model_comparison_loo.csv"))
write_csv(loo_comp_df, p("results/model_comparison/model_comparison_loo_compare.csv"))

cat("\nSaved: model_comparison_dic.csv, model_comparison_waic.csv,\n")
```

```
##
## Saved: model_comparison_dic.csv, model_comparison_waic.csv,
```

```
cat("       model_comparison_loo.csv, model_comparison_loo_compare.csv\n")
```

```
##        model_comparison_loo.csv, model_comparison_loo_compare.csv
```

```
gelman_results <- readRDS(p("results/mcmc/used_car_gelman_results.rds"))
print(gelman_results)
```

```
## Potential scale reduction factors:
##
##             Point est. Upper C.I.
## alpha[1]             1       1.01
## alpha[2]             1       1.01
## alpha[3]             1       1.00
## alpha[4]             1       1.01
## alpha[5]             1       1.01
## alpha[6]             1       1.01
## alpha[7]             1       1.01
## alpha[8]             1       1.01
## alpha[9]             1       1.01
## alpha[10]            1       1.01
## alpha[11]            1       1.01
## alpha[12]            1       1.01
## alpha[13]            1       1.00
## alpha[14]            1       1.00
## alpha[15]            1       1.00
## alpha[16]            1       1.01
## alpha[17]            1       1.01
## alpha[18]            1       1.01
## alpha[19]            1       1.00
## alpha[20]            1       1.01
## alpha[21]            1       1.00
## alpha[22]            1       1.00
## alpha[23]            1       1.00
## alpha[24]            1       1.01
## alpha[25]            1       1.00
## alpha[26]            1       1.00
```

```
## alpha[27]            1       1.01
## alpha[28]            1       1.01
## alpha[29]            1       1.01
## alpha[30]            1       1.00
## alpha[31]            1       1.00
## alpha[32]            1       1.01
## alpha[33]            1       1.00
## alpha[34]            1       1.00
## alpha[35]            1       1.00
## beta[1]              1       1.00
## beta[2]              1       1.00
## beta[3]              1       1.01
## beta[4]              1       1.01
## beta[5]              1       1.00
## beta[6]              1       1.00
## beta[7]              1       1.00
## beta[8]              1       1.00
## beta[9]              1       1.00
## beta[10]             1       1.00
## beta[11]             1       1.00
## beta[12]             1       1.00
## beta[13]             1       1.01
## beta[14]             1       1.00
## mu_alpha             1       1.00
## sigma                1       1.00
## sigma_alpha          1       1.00
```

```
# Step 3: Posterior prediction and OLS comparison


library(tidyverse)
library(coda)

# 1. Load saved objects

inputs <- readRDS(p("data/processed/used_cars_model_inputs.rds"))
samples <- readRDS(p("results/mcmc/used_car_jags_samples.rds"))

train_data <- inputs$train_data
test_data <- inputs$test_data
X_train <- inputs$X_train
X_test <- inputs$X_test
scaling_info <- inputs$scaling_info
brand_levels <- inputs$brand_levels

# y_mean was used to center log_price during cleaning
y_mean <- scaling_info$value[scaling_info$parameter == "y_mean"]

# Convert MCMC samples to matrix
sample_matrix <- as.matrix(samples)

cat("Number of posterior draws:", nrow(sample_matrix), "\n")
```

```
## Number of posterior draws: 6000
```

```r
cat("Number of test observations:", nrow(test_data), "\n")
```

```
## Number of test observations: 785
```

```r
# 2. Extract posterior samples

n_pred <- ncol(X_test)
J <- length(brand_levels)

beta_cols <- paste0("beta[", 1:n_pred, "]")
alpha_cols <- paste0("alpha[", 1:J, "]")

beta_samples <- sample_matrix[, beta_cols, drop = FALSE]
alpha_samples <- sample_matrix[, alpha_cols, drop = FALSE]
sigma_samples <- sample_matrix[, "sigma"]

# 3. Posterior predictive mean for test set

n_test <- nrow(test_data)
n_draws <- nrow(sample_matrix)

pred_log_price_draws <- matrix(NA, nrow = n_draws, ncol = n_test)

for (s in 1:n_draws) {
  fixed_part <- as.vector(X_test %*% beta_samples[s, ])
  brand_part <- alpha_samples[s, test_data$brand_id]
  mu_centered <- brand_part + fixed_part
    mu_log_price <- mu_centered + y_mean

  pred_log_price_draws[s, ] <- rnorm(
    n = n_test,
    mean = mu_log_price,
    sd = sigma_samples[s]
  )
}

pred_price_draws <- exp(pred_log_price_draws)

# Posterior predictive mean
bayes_pred_price <- colMeans(pred_price_draws)

# 95% posterior predictive intervals
bayes_pred_lower <- apply(pred_price_draws, 2, quantile, probs = 0.025)
bayes_pred_upper <- apply(pred_price_draws, 2, quantile, probs = 0.975)

# 4. Bayesian model prediction metrics

actual_price <- test_data$price

rmse <- function(actual, predicted) {
  sqrt(mean((actual - predicted)^2))
}

mae <- function(actual, predicted) {
```

```
  mean(abs(actual - predicted))
}

bayes_rmse <- rmse(actual_price, bayes_pred_price)
bayes_mae <- mae(actual_price, bayes_pred_price)

cat("\nBayesian hierarchical model performance:\n")
```

```
##
## Bayesian hierarchical model performance:
```

```
cat("RMSE:", bayes_rmse, "\n")
```

```
## RMSE: 24052.79
```

```
cat("MAE :", bayes_mae, "\n")
```

```
## MAE : 13429.06
```

```
coverage_95 <- mean(actual_price >= bayes_pred_lower & actual_price <= bayes_pred_upper)

cat("95% predictive interval coverage:", coverage_95, "\n")
```

```
## 95% predictive interval coverage: 0.943949
```

```
# 5. OLS baseline model

ols_model <- lm(
  log_price ~ z_age + z_log_mileage +
    fuel_type + transmission + accident + clean_title + brand_group,
  data = train_data
)

ols_pred_log_price <- predict(ols_model, newdata = test_data)

ols_pred_price <- exp(ols_pred_log_price)

ols_rmse <- rmse(actual_price, ols_pred_price)
ols_mae <- mae(actual_price, ols_pred_price)

cat("\nOLS model performance:\n")
```

```
##
## OLS model performance:
```

```
cat("RMSE:", ols_rmse, "\n")
```

```
## RMSE: 23057.4
```

```
cat("MAE :", ols_mae, "\n")
```

```
## MAE : 13013.17
```

```
# 6. Comparison table

comparison_table <- tibble(
  Model = c("OLS regression", "Bayesian hierarchical model"),
  Test_RMSE = c(ols_rmse, bayes_rmse),
  Test_MAE = c(ols_mae, bayes_mae)
```

```
)

print(comparison_table)
```

```
## # A tibble: 2 x 3
##   Model                       Test_RMSE Test_MAE
##   <chr>                           <dbl>    <dbl>
## 1 OLS regression                 23057.   13013.
## 2 Bayesian hierarchical model    24053.   13429.
```

```
write_csv(comparison_table, p("results/prediction/model_prediction_comparison.csv"))

# 7. Save prediction results

prediction_results <- test_data %>%
  mutate(
    actual_price = actual_price,
    bayes_pred_price = bayes_pred_price,
    bayes_pred_lower = bayes_pred_lower,
    bayes_pred_upper = bayes_pred_upper,
    ols_pred_price = ols_pred_price
  )

write_csv(prediction_results, p("results/prediction/used_car_prediction_results.csv"))

# 8. Basic prediction plot

pdf(p("results/prediction/actual_vs_predicted_bayes.pdf"), width = 7, height = 6)

plot(
  actual_price, bayes_pred_price,
  xlab = "Actual Price",
  ylab = "Bayesian Predicted Price",
  main = "Actual vs Predicted Used Car Prices: Bayesian Hierarchical Model",
  pch = 16,
  col = rgb(0, 0, 0, 0.35)
)

abline(0, 1, lwd = 2)

dev.off()
```

```
## pdf
##   2
```

```
pdf(p("results/prediction/actual_vs_predicted_ols.pdf"), width = 7, height = 6)

plot(
  actual_price, ols_pred_price,
  xlab = "Actual Price",
  ylab = "OLS Predicted Price",
  main = "Actual vs Predicted Used Car Prices: OLS Regression",
  pch = 16,
  col = rgb(0, 0, 0, 0.35)
)
```

```r
abline(0, 1, lwd = 2)

dev.off()
```

```
## pdf
##   2
```

```r
cat("\nSaved files:\n")
```

```
##
## Saved files:
```

```r
cat("- model_prediction_comparison.csv\n")
```

```
## - model_prediction_comparison.csv
```

```r
cat("- used_car_prediction_results.csv\n")
```

```
## - used_car_prediction_results.csv
```

```r
cat("- actual_vs_predicted_bayes.pdf\n")
```

```
## - actual_vs_predicted_bayes.pdf
```

```r
cat("- actual_vs_predicted_ols.pdf\n")
```

```
## - actual_vs_predicted_ols.pdf
```